



Reproducibility Companion Paper: On Learning Disentangled Representation for Acoustic Event Detection

Lijian Gao¹, Qirong Mao^{1,2*}, Jingjing Chen¹, Ming Dong³, Ratna Chinnam⁴, Lucile Sassatelli⁵, Miguel Romero Rondon⁵, Ujjwal Sharma⁶

*Corresponding authors: mao_qr@ujs.edu.cn

¹School of Computer Science and Communication Engineering, Jiangsu University, Zhenjiang 212013, China

²Jiangsu Engineering Research Center of Big Data Ubiquitous Perception and Intelligent Agricultural Applications, Zhenjiang 2122013, China

³Department of Computer Science, Wayne State University, Michigan 48202, USA

⁴Department of Industrial and Systems Engineering, Wayne State University, Michigan 48202, USA

⁵Université Côte d'Azur, CNRS, I3S, 06900 Sophia Antipolis, France

⁶University of Amsterdam, Amsterdam, The Netherlands

ABSTRACT

This companion paper is provided to describe the major experiments reported in our paper "On Learning Disentangled Representation for Acoustic Event Detection" published in ACM Multimedia 2019. To make the replication of our work easier, we first give an introduction of the computing environment where all of our experiments are conducted. Furthermore, we provide an environmental configuration file to setup the compiling environment and other artifacts including the source code, datasets and the files generated during our experiments. Finally, we summarize the structure and usage of the source code. For more details, please consult the README file in the archive of artifacts on GitHub:

https://github.com/mastergofujs/SED_PyTorch.

CCS CONCEPTS

• Computing methodologies → Artificial intelligence.

KEYWORDS

acoustic event detection, disentangled latent representation, supervised variational autoencoder, reproducibility

ACM Reference Format:

Lijian Gao¹, Qirong Mao^{1,2*}, Jingjing Chen¹, Ming Dong³, Ratna Chinnam⁴, Lucile Sassatelli⁵, Miguel Romero Rondon⁵, Ujjwal Sharma⁶. 2021. Reproducibility Companion Paper: On Learning Disentangled Representation for Acoustic Event Detection. In *Proceedings of the 29th ACM International Conference on Multimedia (MM '21)*, October 20–24, 2021, Virtual Event, China. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3474085.3477938>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MM '21, October 20–24, 2021, Virtual Event, China

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8651-7/21/10...\$15.00

<https://doi.org/10.1145/3474085.3477938>

1 ENVIRONMENT INSTALLATION AND CONFIGURATION

All the experiments reported in our prior work of paper [2] are conducted using Ubuntu 16.04 LST, Python 3.6.2 with Pytorch 1.1.0. To run them with GPU, the CUDA 9.0 and CuDNN 7.0 are needed. We provide a configuration file (`py36torch.yaml`) in the root path that lists all the Python libraries needed and software dependencies in our experiments. An Anaconda environment with the required dependencies can be created to run our experiments:

```
conda env create -f py36torch.yaml
```

In the following, we provide the *url* address for downloading all the artifacts associated with our work:

- (1) The datasets and the weights of the pre-trained model can be downloaded from the shared folder on Google Drive. And please note that all pretrained models are provided through the link. If one wants to skip the training stage and use these models directly, they should be put into a proper location in file directories, which will be detailed in the next section. The link is:
<https://drive.google.com/file/d/14NeoqzSHsxljr4B0MiuTixexbZff42cF/>
- (2) The source code and documents can be downloaded from GitHub:
https://github.com/mastergofujs/SED_PyTorch

2 SOURCE CODE STRUCTURE

To make the replication of our experiments easier, we now give a detailed description about the structure and usage of our source code.

As shown in Fig. 1, directory `SED_PyTorch/` is made up of three folders: 1) `aed_data/` saves datasets, the files generated during experiments (e.g., extracted features and weights) and experiment results; 2) `MainClasses/` provides core files for the implementation of our algorithms: Supervised β -VAE, which are packaged into classes; 3) `task_scripts/` contains Python scripts for the major experiments conducted in our study. More specifically:

- 1). `SED_PyTorch/aed_data/`: This folder contains two database directories of the datasets used in our study (i.e., Freesound [1] and TUT [3]), together with the experiment results on these two

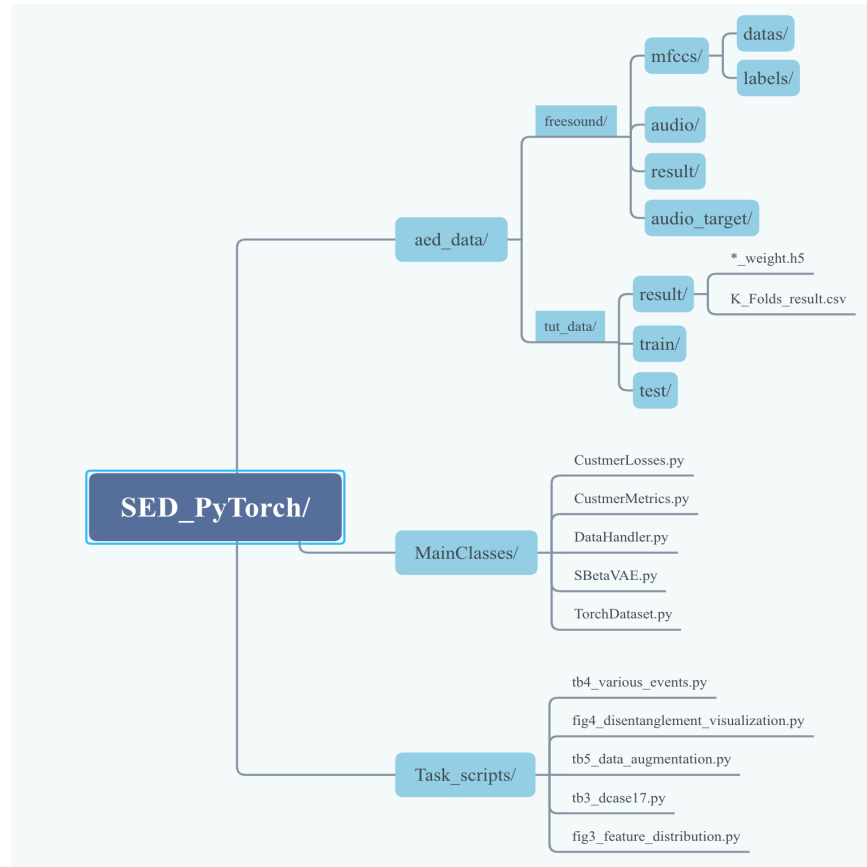


Figure 1: The layout of source code.

datasets. Under the database directories of `freesound/` and `tut_data/`, there are two folders `mfccs/` and `result/`, as shown in Fig. 1. `mfccs/` is used to store the input raw data, and `result/` saves the experimental results on the corresponding database and the weights of the trained model in the K -folds evaluation process. The detailed folder structure of `aed_data/` is given as follows:

- (1) `freesound/`: This folder includes four major sub-directories: `audio/`, `mfccs/`, `result/` and `audio_target/`, all of which are related to the experiments on the Freesound database. We first cut the long audio signal in `audio_target/` as short segments via Audacity (a free software¹) and delete the unclear audio segments. Then, we save the remaining segments into `audio/`. The MFCCs features extracted from the segments in `audio/` are saved in `mfccs/datas/`, and their label files are saved in `mfccs/labels/`. Here, the labels of sound segments are the same as the labels of the long audio signal that they are cut from.
- (2) `tut_data/`: This folder provides a uniform testing set while choosing the training set and the validation set according to the K -folds setup file `train_val_config.json`. Note that, in order to make a fair comparison with other methods, the features saved in the folder `train/mfccs/` and `test/mfccs/`

are extracted using the DCASE Baseline System² [1]. These features are fixed in our experiments.

2). `SED_PyTorch/MainClasses/`: This folder contains several Python class files, which define data processing, model initialization, training and evaluation processes. The relationship between these classes and task scripts are shown in Fig. 2. The details about each class are given below:

- (1) `DataHandler.py`: This Python script defines the class of data processing, which contains an initialization function, several methods for data pre-processing and some important functions, such as `mix_data()`, `load_data()`, etc. Specifically, the `mix_data()` is used to generate sub-dataset with various polyphonic levels, which will be used when evaluating the Freesound dataset with our proposed model. The `load_data()` provides the method to load dataset with various parameters. Note that, the most important parameter of `load_data()` is `fold`, which is used to generate the training and validation sets. It is also worth mentioning that the method `mix_data()` is essential for the Freesound dataset because it is used to generate the sub-dataset with various polyphonic levels. Usage of `mix_data()` to generate polyphonic sound instances is given in the `README` file with details.

¹<https://www.audacityteam.org>

²<https://github.com/TUT-ARG/DCASE2017-baseline-system>

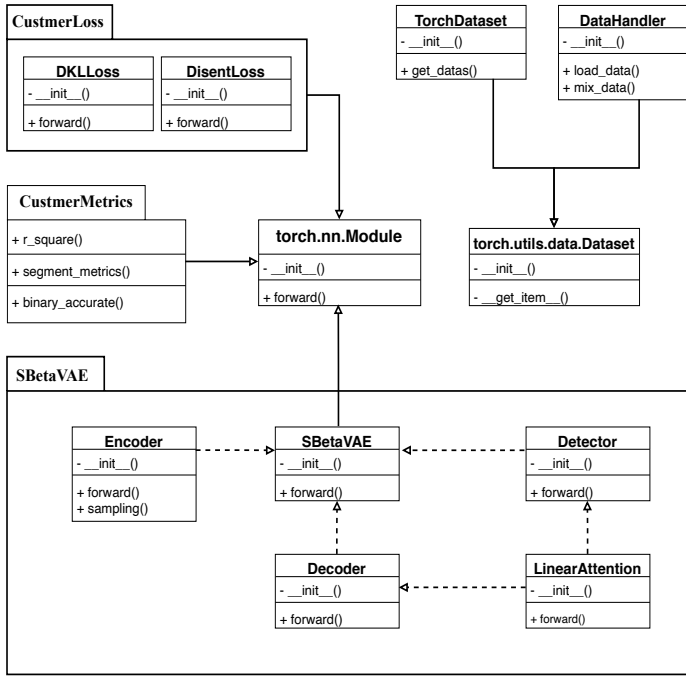


Figure 2: The UML class diagram in source code.

- (2) `TorchDataset.py`: This Python script defines the class of data packing, extending the super class `Dataset` imported from PyTorch. As the data in the array form can not be loaded directly by PyTorch modules, we converted the form of data into Tensor and packed them into the model. For this purpose, we re-write the initialization and `get_item()` methods in the class `TorchDataset`.
 - (3) `CustmerMetrics.py`: According to the DCASE 2017 SED challenge [3], we should apply the two segment-based metrics (segment based F1 score and Error Rate) to evaluate the sound event detection performance. Therefore, in this script, we define the two segment-based metrics for the performance evaluation. Additionally, in order to closely track the training, R Square and Binary Accurate are also defined in `CustmerMetrics.py`.
 - (4) `CustmerLosses.py`: In our prior work, we proposed a novel disentanglement loss function, which constraints the disentanglement in a supervised manner. We implemented the disentanglement loss function in the `CustmerLosses.py` script by extending the `nn.Module` to `CustmerLosses`.
 - (5) `SBetaVAE.py`: This file defines the classes that implement Supervised β -VAE. Specifically, we extend the PyTorch super class `nn.Module` by re-writing the `init()` and `forward()` functions. `SBetaVAE.py` contains five classes: `LinearAttention`, `Encoder`, `Decoder`, `Detector` and `SBetaVAE`. The first four classes define the key layers of the proposed `SBetaVAE` model which is defined in `SBetaVAE` class.
- 3). `SED_PyTorch/task_scripts/`: This folder contains Python scripts provided for the major experiments conducted in our prior work. **It is important to review the code carefully for the**

replication of our work. We provide five script files for various tasks in our study, which correspond to the evaluation of feature distributions (shown in Fig. 3. Note that all the figures and tables mentioned here and below refer to our prior work [2]), disentanglement evaluation (shown in Fig. 4), evaluation experiments on the TUT dataset of DCASE2017 challenge (shown in Table 3), evaluation experiment on Freesound dataset with various event polyphonic levels (shown in Table 4) and evaluation experiment on data augmentation (shown in Table 5), respectively. To make these script files easier to read, we encapsulate each script with at least two functions: `setup_args()` for arguments setup and `running()` for preparing input data, building the model and executing the training/evaluation process. Besides, some of the scripts contain `validation()` and `test()` functions which will not be executed individually but will be called in the `running()` function. All the task scripts can be executed using the following shell command:

```
cd SED_PyTorch/task_scripts/ && python any_script.py [args]
```

where the optional arguments can be found in `setup_args()` or `-h` in terminal and note that, for all experiments, this shell command will execute all the training, validation and testing stages. To execute the testing stage only, the arguments `e` (epoch) should be set to 0 so that no training will be performed.

Although the experiments can be repeated with just one command, we still provide a detailed explanation for each task script in case one needs to run each experiment separately. **It is important to emphasize that one needs to execute these task scripts following the order given below as later task scripts may depend on the model trained in the previous script.**

Algorithm 1 Disentanglement Visualization

Require: $S(t) = [s(t-n), s(t-n+1), \dots, s(t)]$: input signal

Ensure: reconstructed MFCCs $X(t)'$; hot figure

- 1: build the model object and load weights;
- 2: get the latent factors z^* ;
- 3: calculate the *mid*, *max*, and *min* values of z^* ;
- 4: adjust the latent variables from *min* to *max* while fixing others in z^* ;
- 5: decode z^* to reconstruct MFCCs $\hat{X}(mid)$, $\hat{X}(min)$ and $\hat{X}(max)$ features;
- 6: calculate the *delta* map among the generated $\hat{X}$;
- 7: plot the *delta* map as the hot figure.

Algorithm 2 Data Augmentation

Require: $S(t) = [s(t-n), s(t-n+1), \dots, s(t)]$: input signal

Ensure: augmented data

- 1: construct unbalanced sub-dataset by Freesound dataset;
- 2: train and evaluate the model with the unbalanced dataset;
- 3: get the latent factors z^* for each event category;
- 4: decode z_k^* to reconstruct MFCCs features for the event category which is trained with the least training data.
- 5: retrain the model with the additional reconstructed features and evaluate it.

Algorithm 3 Feature distribution**Require:** $S(t) = [s(t - n), s(t - n + 1), \dots, s(t)]$: input signal**Ensure:** 2-dimensional vectors; distribution figures;

- 1: build the model object and load weights;
- 2: extract output h_m of the m -th hidden layers with n samples;
- 3: create t -SNE object and initialize with PCA;
- 4: train the t -SNE model and transform high-level features h_m into 2 dimension ones;
- 5: plot hot figure with the transformed data.

- (1) `tb4_various_event.py`: This script evaluates the performance of our methods on data of various polyphonic levels made from Freesound as shown in Table 4. In this script, there are many optional arguments which will determine the structure of the model. We can conduct this experiment after we choose suitable values for these arguments. For example, if we want to train our model with the samples which contain 10 event categories, we need to call the following shell command:

```
python tb4_various_events.py -k 10 -b 128 -e 50 -lr 0.0003 -m 1
```

where b , e , lr , k denote the batch size, the training epochs, the learning rate and the number of categories of sound events respectively. The values of these arguments are set the same as the original paper [2]. Here, **it is important to note that the argument m denotes whether one needs to generate new training data through Freesound dataset**. By default, m is set as 1 (new data will be generated). The data generation can be turned off by setting it as 0. In addition, to reproduce all the results shown in Table 4, the value of the parameter k should be changed from 5 to 20 with an interval of 5. The details of optional arguments can be found in `setup_args()`.

- (2) `fig4_disentanglement_visualization.py`: This script is used to evaluate the disentanglement performance, listed in Fig. 4. To qualitatively show event-specific disentangled factors learned by supervised β -VAE, we need to run the operations in Algorithm 1 by calling the shell command below:
- ```
python fig4_disentanglement_visualization.py
```
- (3) `tb5_data_augmentation.py`: This script gives the method to evaluate data augmentation, listed in Table 5. The detailed steps are given in Algorithm 2 by calling the shell command below:
- ```
python tb5_data_augmentation.py
```
- (4) `tb3_dcse17.py`: This script evaluates the performance of the proposed method on DCASE 2017 SED challenge TUT dataset as shown in Table 3. Since the training and testing set are selected by the DCASE challenge, we only need to specify b (batchsize), e (epoch), and lr (learning rate) when calling the shell command below:
- ```
python tb3_dcse17.py -b 128 -e 50 -lr 0.0003
```
- (5) `fig3_feature_distribution.py`: This script is used to visualize the distributions of the features learned by the proposed model, shown in Fig. 3. The visualization procedure is

dependent on  $t$ -SNE. In general, the distribution visualization can be obtained by Algorithm 3. Finally, to show the figure of feature distribution, call the command below and the figure file (in *PNG* format) will be saved at `aed_data/tut_data/result/d-istribution.png`:

```
python fig3_feature_distribution.py
```

In order to make the replication of our work simpler, we have put all the necessary steps in each algorithm into the `running()` function.

### 3 PRETRAINED MODELS

At last, we give the introduction of the pretrained models provided together with dataset on Google Drive. In the training stage of each experiment, we evaluate the model on the validation set and save model checkpoints for every epoch when we get the minimum loss. Therefore, the pretrained models we provided were generated by the same task scripts mentioned above. For example, the line 132 of the code in `tb3_dcse.py` shows the generation processing: `torch.save([model weights], [path], [file name])`

Finally, to use the model checkpoints for testing, one should copy the pretrained model files to the corresponding directories and set the parameter  $e$  (epoch) as 0, because when training the model, it will generate new model checkpoint and overwrite the pretrained model we provided.

### 4 ACKNOWLEDGEMENT

This work is supported by the Key project of National Nature Science Foundation of China (Grant No. U1836220), the Jiangsu key research and development plan (industry foresight and key core technology, Grant No. BE2020036), the general project of National Nature Science Foundation of China (Grant No. 61672267) and the Qing Lan Talent Program of Jiangsu Province, all to Qirong Mao.

### REFERENCES

- [1] Frederic Font, Gerard Roma, and Xavier Serra. 2013. Freesound technical demo. In *Proceedings of the 21st ACM international conference on Multimedia*. ACM, October, 17–21, 2013. Barcelona, Spain, 411–412.
- [2] Lijian Gao, Qirong Mao, Ming Dong, Yu Jing, and Ratna Chinnam. 2019. On Learning Disentangled Representation for Acoustic Event Detection. In *Proceedings of the 27th ACM International Conference on Multimedia*. ACM, October, 21–25, 2019. Nice, France, 2006–2014.
- [3] Annamaria Mesaros and Toni Heittola. 2017. DCASE 2017 Challenge Setup: Tasks, datasets and baseline system. In *Workshop on Detection and Classification of Acoustic Scenes and Events*.